

Day 4 — OOP & Reusable Code

Classes, Objects & Building a Context Manager — Notes for Recap

1. Classes

What is a class?

A class is a blueprint for creating objects — it defines what data (attributes) and behavior (methods) something will have, without yet being a real, usable thing.

```
class Pipeline:
    def __init__(self, name):
        self.name = name

    def run(self):
        print(f"Running pipeline: {self.name}")
```

Key takeaway: A class defines the structure once; you can then create as many objects from it as needed, each with its own data.

2. Objects

An object is a specific instance created from a class — the actual, usable thing, with its own values filled in.

```
sales_pipeline = Pipeline("Sales ETL")
inventory_pipeline = Pipeline("Inventory ETL")

sales_pipeline.run()      # Running pipeline: Sales ETL
inventory_pipeline.run()  # Running pipeline: Inventory ETL
```

Key takeaway: The class is the blueprint; the object is the actual house built from it. You can build many different objects from the same class, each independent of the others.

3. Inheritance

Inheritance lets one class reuse and extend the behavior of another class, instead of rewriting the same code.

```

class Pipeline:
    def __init__(self, name):
        self.name = name

    def run(self):
        print(f"Running pipeline: {self.name}")

class StreamingPipeline(Pipeline):  # inherits from Pipeline
    def run(self):
        print(f"Streaming pipeline running continuously: {self.name}")

sp = StreamingPipeline("Clickstream")
sp.run()  # Streaming pipeline running continuously: Clickstream

```

Key takeaway: *Inheritance avoids duplicating code between similar classes — a StreamingPipeline can reuse everything from Pipeline and only override what's different.*

4. Encapsulation

Encapsulation means keeping an object's internal details protected, and only exposing what's necessary through defined methods — preventing code elsewhere from directly messing with internal state in unexpected ways.

```

class BankAccount:
    def __init__(self, balance):
        self._balance = balance  # underscore = "internal, please don't touch directly"

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def get_balance(self):
        return self._balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())  # 1500

```

Key takeaway: *Encapsulation protects data integrity — changes to `_balance` happen only through controlled methods like `deposit()`, not by random code changing it directly and possibly breaking assumptions elsewhere.*

5. `__init__`

`__init__` is a special method that runs automatically when a new object is created from a class. It's used to set up the object's initial data.

```
class DataSource:
    def __init__(self, source_name, file_path):
        self.source_name = source_name
        self.file_path = file_path
        print(f"Initialized data source: {self.source_name}")

ds = DataSource("Sales CSV", "data/sales.csv")
# Output: Initialized data source: Sales CSV
```

Key takeaway: `__init__` runs the moment an object is created — it's the natural place to set up anything the object needs before it's used (file paths, connection details, default settings).

6. Practice: A Database-Connection Context Manager

A context manager (used with the `with` keyword, seen on Day 2 for file handling) guarantees that setup and cleanup code always run — even if an error happens in between. Building a class-based one is today's hands-on practice.

```
class DatabaseConnection:
    def __init__(self, db_name):
        self.db_name = db_name

    def __enter__(self):
        print(f"Opening connection to {self.db_name}")
        return self # this is what 'as conn' will receive

    def __exit__(self, exc_type, exc_value, traceback):
        print(f"Closing connection to {self.db_name}")
        # returning False lets any error still be raised normally
        return False

# Using it:
with DatabaseConnection("sales_db") as conn:
    print("Running a query...")

# Output:
# Opening connection to sales_db
# Running a query...
# Closing connection to sales_db
```

How it works: `__enter__` runs when the `with` block starts (open the connection), and `__exit__` runs automatically when the block ends — whether it finished normally or an error occurred. This guarantees the connection is always closed properly.

Key takeaway: This is the exact same safety pattern as with `open(...)` from Day 2, but built by hand. Real database libraries (like `psycopg2`) use this same pattern internally.

Day 4 Summary — OOP & Reusable Code

Concept	One-Line Recap
Classes	Blueprints that define data and behavior
Objects	Actual instances created from a class, each with its own data
Inheritance	Reusing and extending another class's behavior instead of duplicating code
Encapsulation	Protecting internal data, exposing only controlled access through methods
<code>__init__</code>	Special method that sets up an object's data the moment it's created
Context Manager	A class using <code>__enter__</code> / <code>__exit__</code> to guarantee setup and cleanup always run

Overall takeaway: *Don't spend days memorizing every OOP pattern - the goal today was just enough OOP to build one genuinely useful thing: a context manager that safely manages a resource (like a database connection), the same pattern used throughout production-grade Python code.*